

Mobile Robotics - Assignment 1

João Martins, José Carvalho, Lourenço Pinho and Maria Lopes

For the Mobile Robotics course, a differential robot was developed from scratch (Figure 1). For this assignment, it was necessary to weld the hardware parts and required electrical connections. To correctly control the robot's movement, the intrinsic parameters and dynamic model of the robot's DC motors¹ (Figure 2) needed to be determined. In this assignment, the formulation of a control system for the motor used will be presented. It will be assumed that the motors share the same dynamical model, and considerations on the kinematic model will be left to the next assignment.

To determine the intrinsic parameters, tests were conducted to measure the current and angular velocity of the motor. Two methods can be used to estimate the parameters from the collected data: (i) linear regression of the collected data and (ii) least squares method. Both were applied, and the results were later compared.

Finally, the dynamic model of the motor was obtained by applying a voltage step outside the motor's dead zone and recording the corresponding angular velocities. Subsequently, these data points determined the gain and time constant of the first-order equation that represents the model.

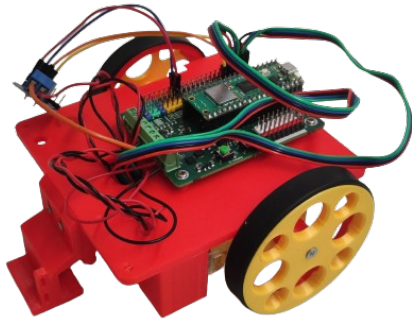


Figure 1: Built robot

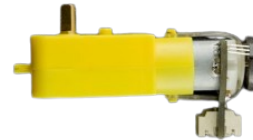


Figure 2: DC motor

1 Motor Intrinsic Parameters Estimation

The intrinsic parameters of the motor are the physical properties of the motor that determine its behavior. These parameters include:

- Internal Resistance (R): The internal resistance of the motor is the resistance of the motor winding to the flow of current. It is measured in ohms (Ω).
- Torque Constant (K): The motor's torque constant is the ratio between the torque generated by the motor and the current flowing through the motor winding. It is measured in ($V/(\text{rad} \cdot s)$).
- Friction Coefficient (B): The motor's friction coefficient is a measurement of the friction between the moving parts of the motor. It is measured in Newton meters times second per radian ($N \cdot m \cdot s/\text{rad}$).

¹<https://www.botnroll.com/pt/motores-dc/1645-micro-motor-dc-com-encoder-e-redutora-1201-6v-160rpm.html>

- Load Torque (T_q): The Torque that the load applies to the Motor Shaft. Unless specified, the tests will be made with no load in this work, and therefore T_q is the Static Torque. In this scenario, it corresponds to the minimum amount of Torque to overcome static friction ($\text{N} \cdot \text{m}$).

The motor can be represented by the schematics in [Figure 3](#). From the Kirchhoff laws and the second Newton Law applied to rotational motion, one can deduce the following system of equations for the system:

$$\begin{cases} v(t) = R \cdot i(t) + L \cdot \frac{di(t)}{dt} + v_b(t) \\ J \cdot \dot{\omega}(t) = T_m(t) - B \cdot \omega(t) - T_q \end{cases}, \text{ where } T_m = K \cdot i \text{ and } v_b = K \cdot \omega \quad (1)$$

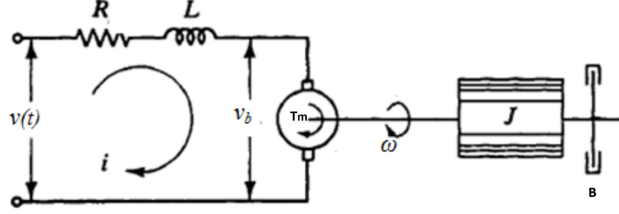


Figure 3: Motor equivalent electric circuit

From [Equation 1](#), considering the system has reached a steady state and therefore is at an equilibrium point ($\frac{di}{dt} = 0$ and $\dot{\omega} = 0$), one can deduce [Equations 2](#) and [3](#), where $u = v(t)$ will be our manipulated input.

$$\frac{u}{\omega} = R \cdot \frac{i}{\omega} + K \quad (2)$$

$$K \cdot i = B \cdot \omega + T_q \quad (3)$$

These equations allow us to estimate the intrinsic values of the motor by applying voltage u to the motor and measuring the current and angular velocity from the encoders. The measuring procedure consisted of two steps. In the first, the motor is preheated for 6 seconds by setting the voltage to 5 V to reduce parameter variation in the second step. This step is essential not to break the time invariance assumption. The second step is to apply different voltage values $u = [0, 5]$ V with increments of 0.25 V, $u \in \{0, 0.25, \dots, 0.5\}$ V. 50 samples of current and angular velocity for each voltage value were recorded. The procedure mentioned above is illustrated in the state machine of [Figure 4](#). [Table 1](#) contains the data obtained from the tests. The voltage is the nominal value applied to the motor, and the current and angular velocity columns contain the average of the samples acquired during the test for each voltage applied to the motor. To calculate the intrinsic parameters the scripts in [Appendix A](#) and [Appendix B](#) were used.

1.1 Linear Regression

With the data present in [Table 1](#), the different plots from [Figure 5](#) and [Figure 6](#) can be obtained.

Looking at the plot from [Figure 5a](#) the dead-zone of the motor can be seen, in this case, point A is still in the dead-zone, so it won't be used in the parameters calculations. On the other hand, from the plot of [Figure 5b](#), we can see that the current is not continuous.

From the plot in [Figure 6a](#) and [Equation 2](#), the values of R and K are obtained as the slope and the intercept, respectively. In this case, $R = 3.3505 \, \Omega$ and $K = 0.3049 \, \text{V}/(\text{rad} \cdot \text{s})$.

Similarly, from the plot of [Figure 6b](#) and [Equation 3](#), the values of B and T_q are obtained as the slope and the intercept, respectively. In this case, $B = 0.0022 \, \text{N} \cdot \text{m} \cdot \text{s}/\text{rad}$ and $T_q = 0.022 \, \text{N} \cdot \text{m}$.

From the plots, it was estimated that the dead zone occurred for $u = 0$ and $u = 0.25$ V, but with increments of 0.25 V it is hard to estimate the real motor's dead zone. From the estimated intrinsic parameters, it is possible to estimate the expected limit value. The voltage can be written as follows:

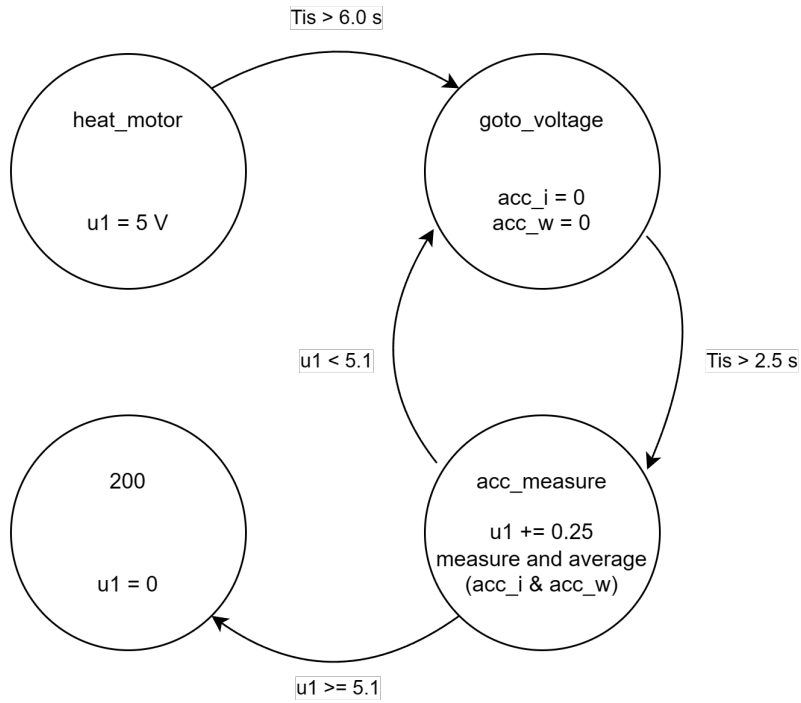


Figure 4: Motor equivalent electric circuit

Table 1: Data collected to estimate intrinsic parameters

Voltage (u)	Current (i)	Angular velocity (ω)
0.000	0.000	0.000
0.250	0.075	0.000
0.500	0.074	0.806
0.750	0.082	1.622
1.000	0.091	2.313
1.250	0.099	3.034
1.500	0.105	3.851
1.750	0.108	4.597
2.000	0.113	5.328
2.250	0.114	6.111
2.500	0.119	6.925
2.750	0.125	7.632
3.000	0.130	8.435
3.250	0.136	9.061
3.500	0.143	9.799
3.750	0.148	10.433
4.000	0.155	11.143
4.250	0.160	11.849
4.500	0.168	12.242
4.750	0.175	13.101
5.000	0.179	13.588

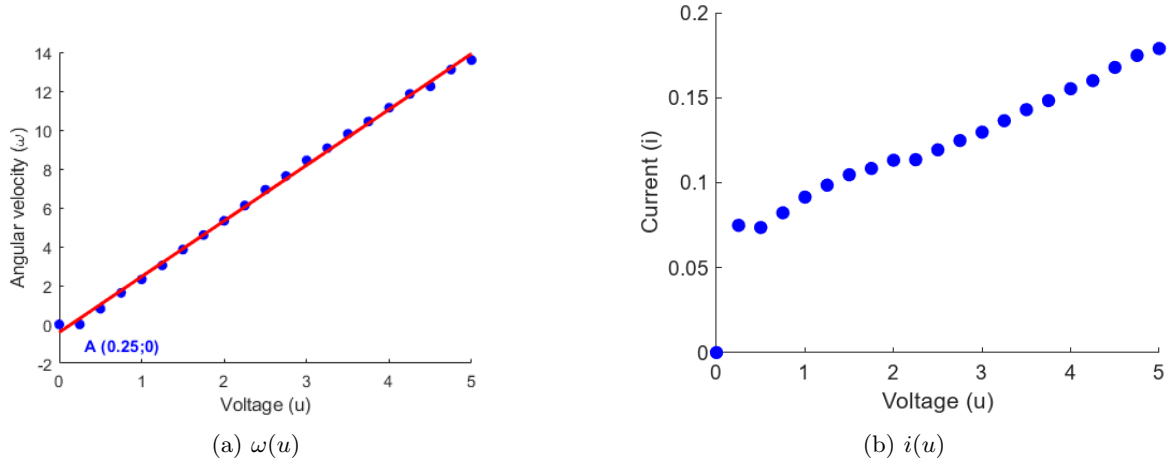


Figure 5: Measured data for deadzone determination

$$\begin{aligned}
 u &= \frac{R}{K}(B \cdot w + T_q) + \frac{k^2 \cdot w}{w} \Leftrightarrow \\
 k \cdot u &= R \cdot B \cdot w + R \cdot T_q + k^2 \cdot w \Leftrightarrow \\
 w(R \cdot B + k^2) &= K \cdot u - R \cdot T_q
 \end{aligned} \tag{4}$$

And so, the dead zone limit value can be computed from Equation 5.

$$\begin{aligned}
 K \cdot u - R \cdot T_q &= 0 \Leftrightarrow \\
 k \cdot u &= R \cdot T_q \Leftrightarrow \\
 u &= \frac{R \cdot T_q}{k} \Leftrightarrow \\
 u &= \frac{3.2064 \cdot 0.0224}{0.3063} = 0.2345\text{V}
 \end{aligned} \tag{5}$$

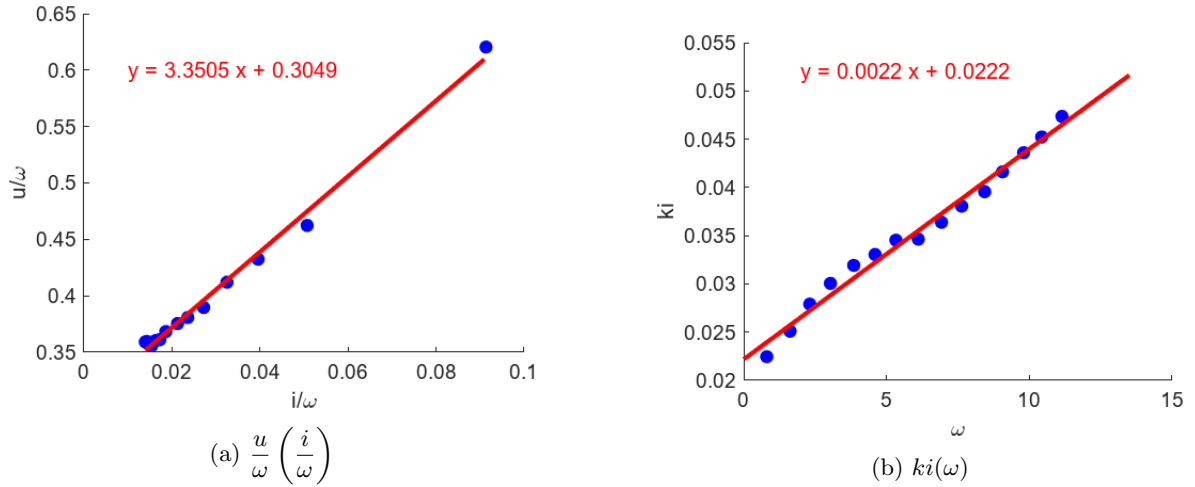


Figure 6: Linear regressions of the measured data.

1.2 Least squares method

When determining the intrinsic parameters of a DC motor using the least squares method, it is essential to consider that this method does not amplify the existing noise in the system, unlike the linear regression method described in section 1.1. By applying the least squares method, the analysis focuses

on minimizing the sum of the squares of the differences between the observed values and those predicted by the model.

$$Y = \theta^T X \Leftrightarrow \theta = (X^T X)^{-1} X^T Y \quad (6)$$

The values obtained from this method are the following: $R = 3.2064 \, \Omega$, $K = 0.3063 \, \text{V}/(\text{rad} \cdot \text{s})$, $B = 0.0022 \, \text{N} \cdot \text{m} \cdot \text{s}/\text{rad}$ e $T_q = 0.0224 \, \text{N} \cdot \text{m}$, where $y = u$, $\theta = [R \quad K]$ and $x = [i \quad w]$ for Equation 2 and $y = K \cdot i$, $\theta = [B \quad T_q]$ and $x = [w \quad 1]$ for Equation 3.

2 Motor dynamics model

Up until this section, the tests focused on determining intrinsic parameters that will be used to simulate the system in SimTwo. This section will focus on determining the dynamic model of the system. The objective is to determine a Single-Input Single-Output (SISO) linear transfer function by using the system of differential equations in 1. By applying the Laplace Transform, one can deduce the following expression in the s -domain:

$$\begin{cases} U(s) &= (R + sL)I(s) + KW(s) \\ sJW(s) &= KI(s) - BW(s) - T_q \end{cases} \Leftrightarrow \begin{cases} U(s) &= (R + sL)I(s) + KW(s) \\ KI(s) &= (sJ + B)W(s) - T_q \end{cases} \quad (7)$$

Note that it is not possible to get a linear transfer function $\frac{I(s)}{W(s)}$, as the expression is affine due to the term $-T_q$. To get a linear expression, we will consider that T_q is equal to zero and then compensate for this nonlinearity with a Hammerstein Control Block. Therefore, we can continue our deduction as:

$$\begin{aligned} (R + sL)I(s) + KW(s) &= U(s) \Leftrightarrow \\ \frac{1}{K}(R + sL)(sJ + B)W(s) &= U(s) \Leftrightarrow \\ \frac{W(s)}{U(s)} &= \frac{K}{(R + sL)(sJ + B)} \end{aligned} \quad (8)$$

Resulting in a second-order system. In this scenario, the pole from the electrical part $s = \frac{-R}{L}$ is typically much smaller than the pole $s = \frac{-B}{J}$. To simplify the estimation and, later the control process, the final assumption is that this system can be approximated as a first-order transfer function:

$$\frac{W(s)}{U(s)} = \frac{K}{sJ + B} \Leftrightarrow \frac{A}{\tau s + 1} \quad (9)$$

To estimate the gain A and the time constant τ , we applied a voltage step from $I_1 \, \text{V}$ to $I_2 \, \text{V}$ and observed the step response. This test was done with load (the robot was in contact with the ground) and without load. Table 2 also contains the data for both tests. For the case with load $I_1 = 0.5 \, \text{V}$ and $I_2 = 2 \, \text{V}$. As for the other scenario, $I_1 = 0.5 \, \text{V}$ and $I_2 = 2 \, \text{V}$ ².

Considering that the time constant can be approximated as the time the system takes to reach 63.2% of the final output value $O2$ from the input value $O1$. The transfer function parameters for the system with load can be approximated as:

$$A = \frac{\Delta O}{\Delta I} = 2.9725 \, \text{rad}/(\text{s} \cdot \text{V}) \quad (10)$$

$$O(63, 2\%) = \omega_\tau = 0.632 \cdot (O2 - O1) + O2 = 3.595 \, \text{rad/s} \quad (11)$$

²We acknowledge that the test should be done with the same Voltage signal. We did not have time to repeat the test, and since the motor is operating in its linear region, the difference caused by this error should be minimal.

Table 2: Measured data

Time (t)	Angular velocity (ω)	
	Without load 0.5 V $\rightarrow$ 4.5 V	With load 0.5 V $\rightarrow$ 2 V
0.000	0.777	0.777
0.040	2.127	1.268
0.080	4.827	2.291
0.120	6.790	3.027
0.160	8.263	3.600
0.200	9.368	3.968
0.240	10.145	4.336
0.280	10.717	4.500
0.320	11.126	4.745
0.360	11.454	4.827
0.400	11.699	4.950
0.440	11.863	5.031
0.480	12.026	5.072
0.520	12.108	5.113
0.560	12.149	5.154
0.600	12.231	5.154
0.640	12.272	5.195
0.680	12.272	5.195
0.720	12.272	5.195
0.760	12.231	5.236
0.800	12.231	5.195

A crucial step in the estimation of the robot's dynamic model was the determination of the time constant of the DC motor (τ). This involves collecting motor response data at different control inputs. Then, using techniques like linear interpolation, the relationship between control input and the time to reach a specific response level (63% of steady-state speed) is used to estimate τ based on the chosen data points.

$$\tau = \frac{(\omega_\tau - \omega_1) \cdot (\omega_2 - \omega_1)}{t_2 - t_1} + t_1 = 0.1596 \text{ s} \quad (12)$$

From the estimated parameters, we can deduce the approximated value function $G(s)$:

$$G(s) = \frac{W(s)}{U(s)} = \frac{3.595}{0.1596 \cdot s + 1} \quad (13)$$

As for the test without load, applying the same calculations, the results were $\tau = 0.1512 \text{ s}$ and $A = 2.83 \text{ rad}/(s \cdot V)$. It is visible that, as expected, the system with load is slightly slower and has a larger gain. While the difference between models is small, for the rest of this document and the course, we will consider the model with load, presented in equation (13), as the robot will operate in conditions more similar to this test.

To validate this value function, the approximated system was simulated in Matlab using the Control System Toolbox (code in [Appendix C](#)). A step reference of 1.5 V was applied to the system. To compensate for the non-linearity of the real system, the offset $w' = 0.77$ rad/s was added to the output of the simulated system. The comparison between step responses can be visualized in Figure 7.

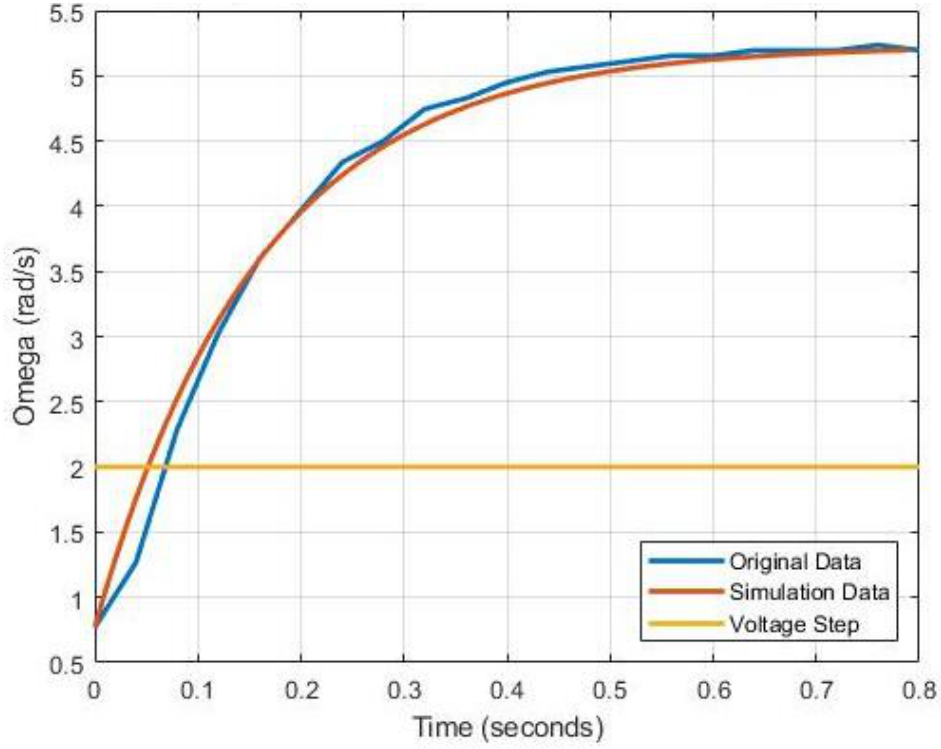


Figure 7: Comparison of Original Data and Simulation Output.

From the visualization, it can be inferred that the approximated value function adequately captures the dynamics of the real system. However, it is important to note that the simulated system is slightly faster when compared with real data due to the first-order approximation. This is especially visible from $t = [0, 0.10]$ s, where the effect from the pole $s = -\frac{R}{L}$ is most present, but as time passes, the corresponding exponential decays quicker, and the effect is reduced. It is also interesting to point out that at $t \approx 0.16$ s $\approx \tau$, the output of both systems is identical to the result of equation (11), once again validating our approximation.

3 Conclusion

After building the robot, the first step besides testing must be extracting the information required for the low-level firmware. The low-level must be consolidated to achieve good performance and accuracy in the higher-level programming blocks. Extracting the motor's intrinsic parameters allows higher wheel velocity control and better response. Furthermore, the extracted parameters can adjust the simulator's response to match the real robot. This way, validating the implementations on a simulator that includes the sensor readings and using the robot's microcontroller with hardware in the loop will be possible. This way, the robot's behavior can be replicated in the simulator, making it easier to debug and test higher-level software modules, such as path planning or localization.

A Intrinsic parameters estimation - Matlab

```
clear all;

dataTable = readtable('Data.csv');
data = table2array(dataTable);

u = data(:, 1); % Column 1 for 'u'
i = data(:, 2); % Column 2 for 'i'
w = data(:, 3); % Column 3 for 'w'

%% Linear Regression

% w(u)
coefficients = polyfit(u(1:end), w(1:end), 1);
m_wu = coefficients(1);
b_wu = coefficients(2);

figure;
subplot(1,2,1);
scatter(u, w, 'b', 'filled');
hold on;
xlabel('Voltage (u)');
ylabel('Angular velocity (\omega)');
text(0.3, -1, 'A (0.25;0)', 'FontSize', 10, 'FontWeight', 'bold', 'Color', 'b')
grid on;

u_fit = min(u):0.1:max(u);
w_fit = m_wu * u_fit + b_wu;
plot(u_fit, w_fit, 'r', 'LineWidth', 2);
hold off;

% i(u)
coefficients = polyfit(u, i, 1);
m_iu = coefficients(1);
b_iu = coefficients(2);
subplot(1,2,2);

scatter(u, i, 'b', 'filled');
xlabel('Voltage (u)');
ylabel('Current (i)');
grid on;

% u/w(i/w)
u_w = u(3:end-4)./w(3:end-4);
i_w = i(3:end-4)./w(3:end-4);
coefficients = polyfit(i_w, u_w, 1);
m_iwu = coefficients(1);
b_iwu = coefficients(2);

figure;
subplot(1,2,1);
scatter(i_w, u_w, 'b', 'filled');
hold on;
xlabel('i/\omega');
ylabel('u/\omega');

iw_fit = min(i_w):0.0005:max(i_w);
uw_fit = m_iwu * iw_fit + b_iwu;
plot(iw_fit, uw_fit, 'r', 'LineWidth', 2);
hold off;

K = b_iwu
R = m_iwu

caption = sprintf('y = %.4f x + %.4f', m_iwu, b_iwu);
text(0.01, 0.6, caption, 'FontSize', 10, 'Color', 'r', 'FontWeight', 'normal');

% ki(w)
ki=K.*i(3:end-4);
coefficients = polyfit(w(3:end-4), ki, 1);
```



```

m_ki = coefficients(1);
b_ki = coefficients(2);

B = m_ki
Tq = b_ki

subplot(1,2,2);
scatter(w(3:end-4), ki, 'b', 'filled');
hold on;
xlabel('\omega');
ylabel('ki');

caption = sprintf('y = %.4f x + %.4f', m_ki, b_ki);
text(2, 0.052, caption, 'FontSize', 10, 'Color', 'r', 'FontWeight', 'normal');

w_fit = min(w):0.1:max(w);
ki_fit = m_ki * w_fit + b_ki;
plot(w_fit, ki_fit, 'r', 'LineWidth', 2);
hold off;

%% Least Squares

% u/w(i/w)
Y_u = u(2:end-5);
X_iw = [i(2:end-5) w(2:end-5)];

theta = lsqr(X_iw,Y_u);
R_ls = theta(1)
K_ls = theta(2)

% ki(w)
Y_ki = K_ls*i(2:end-5);
X_w = [w(2:end-5) ones(length(w)-6,1)];

theta_bt = lsqr(X_w,Y_ki);

B_ls = theta_bt(1)
Tq_ls = theta_bt(2)

```

B Intrinsic parameters - Python

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Given data
data = np.array([
    [0, 0, 0],
    [0.25, 0.074887, 0],
    [0.5, 0.073608, 0.805851],
    [0.75, 0.0822625, 1.62234],
    [1, 0.0914855, 2.31283],
    [1.25, 0.09854, 3.03442],
    [1.5, 0.104665, 3.85091],
    [1.75, 0.108376, 4.59703],
    [2, 0.113232, 5.32843],
    [2.25, 0.113566, 6.11138],
    [2.5, 0.119333, 6.92459],
    [2.75, 0.124807, 7.63227],
    [3, 0.129729, 8.43485],
    [3.25, 0.136471, 9.06071],
    [3.5, 0.142978, 9.79866],
    [3.75, 0.148318, 10.4327],
    [4, 0.15532, 11.1428],
    [4.25, 0.16006, 11.8489],
    [4.5, 0.167848, 12.2424],
    [4.75, 0.174906, 13.1014],
    [5, 0.17898, 13.5882]
])

```

```

# Extracting columns into arrays
u = data[:, 0] # Column 1 for 'u'
i = data[:, 1] # Column 2 for 'i'
w = data[:, 2] # Column 3 for 'w'

## Linear Regression

# i(u)
plt.scatter(u,i)
plt.xlabel('Voltage ( $u$ )')
plt.ylabel('Current ( $i$ )')

plt.show()

# w(u)
plt.scatter(u,w,label='Raw Data')
plt.xlabel('Voltage ( $u$ )')
plt.ylabel('Angular Speed ( $\omega$ )')

regressor_uw = LinearRegression()
u_pred = np.linspace(min(u[1:]), max(u[1:]), 100).reshape(-1, 1)
regressor_uw.fit(u[1:].reshape(-1, 1), w[1:])
w_pred = regressor_uw.predict(u_pred)

plt.plot(u_pred, w_pred, color='red',label='Linear Regression')
plt.text(0.45, -0.25, 'A (0.25;0)', fontsize=12, color='blue')

plt.show()

# u/w(i/w)
y=u[2:-5]/w[2:-5]
x=i[2:-5]/w[2:-5]
plt.scatter(x,y)
plt.xlabel(' $i$/$\omega$ ')
plt.ylabel(' $u$/$\omega$ ')

regressor_uw_iw = LinearRegression()
x_pred = np.linspace(min(x), max(x), 100).reshape(-1, 1)
regressor_uw_iw.fit(x.reshape(-1, 1), y)
y_pred = regressor_uw_iw.predict(x_pred)

plt.plot(x_pred, y_pred, color='red')
slope = regressor_uw_iw.coef_[0]
intercept = regressor_uw_iw.intercept_
equation = f'y = {slope:.4f}x + {intercept:.4f}'
plt.text(0.05, 0.6, equation, fontsize=12, color='red')
plt.show()

R = slope
K = intercept

# ki(w)
ki=K*i[2:-5]
print(ki)
wx=w[2:-5]
print(w)

plt.scatter(wx,ki)
plt.xlabel(' $ki$ ')
plt.ylabel(' $\omega$ ')

regressor_ki_w = LinearRegression()
x_pred = np.linspace(min(wx), max(wx), 100).reshape(-1, 1)
regressor_ki_w.fit(wx.reshape(-1, 1), ki)
y_pred = regressor_ki_w.predict(x_pred)

plt.plot(x_pred, y_pred, color='red')
slope = regressor_ki_w.coef_[0]
intercept = regressor_ki_w.intercept_
equation = f'y = {slope:.4f}x + {intercept:.4f}'

```

```

plt.text(4, 0.04, equation, fontsize=12, color='red')
plt.show()

## Linear Regression

Y = np.array(u[2:-5])
X = np.vstack((i[2:-5], w[2:-5])).T

theta = np.linalg.inv(X.T @ X) @ X.T @ Y

print(theta)

Y = np.array(ki)
X = np.vstack((w[2:-5], np.ones_like(w[2:-5]))).T

theta = np.linalg.inv(X.T @ X) @ X.T @ Y

print(theta)

```

C Dynamic model validation - Matlab

```

omega = [0.77,1.26809,2.29074, 3.02706, 3.59974, 3.9679, 4.33605, 4.49968, 4.74511,
         4.8269 ,4.94964, 5.03146, 5.07236, 5.11327, 5.15418, 5.15418, 5.19508, 5.19508,
         5.19508, 5.23599, 5.19508];

t = [0.0,0.04,0.08, 0.12, 0.16, 0.2, 0.24, 0.28, 0.32, 0.36, 0.4, 0.44, 0.48, 0.52,
     0.56, 0.6, 0.64, 0.68, 0.72, 0.76, 0.8];
s = tf('s');
sys = 2.9725/(s*0.1596+1);

% Define the time vector for simulation
t_ = 0:0.01:5; % Time vector from 0 to 10 seconds with a step size of 0.01

% Define the input signal (step from 0.5 to 2.0)
u = 1.5 * (t_ >= 1.00);

% Simulate the response
[y, t_] = lsim(sys, u, t_);

% Plot both data sets
plot(t,omega, 'DisplayName', 'Original Data','Linewidth',2);
hold on;
plot(t_(101:180)-1,y(101:180)+0.77, 'DisplayName', 'Simulation Data','Linewidth',2);
plot(t,2*ones(21,1), 'DisplayName', 'Voltage Step','Linewidth',2);

% Add grid for better readability
grid on;

xlabel('Time (seconds)');
ylabel('\omega (rad/s)');

% Add legend to differentiate plots
legend('Location', 'southeast');

```